Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Prof. Dr. Patrick Levi

Projektarbeit Machine Learning WiSe 2025/26

Raphael Josef Balzer, Steven Michael Fischer, Kevin Paulus

Studiengang Master Künstliche Intelligenz (MKI)

16. Januar 2026

Inhaltsverzeichnis

1 Einleitung **3**
1.1 Motivation und Problemstellung . . . 3
1.2 Zielsetzung der Arbeit . . . 3
1.3 Aufbau der Arbeit . . . 3

2 Related Work und Theoretische Grundlagen **4**
2.1 Intrusion Detection und Machine Learning . . . 4
2.2 Der TON-IOT-Datensatz . . . 4
2.3 Support Vector Machines für IDS . . . 4
2.4 Evaluationsmetriken für Sicherheitsanwendungen . . . 5
2.5 Benchmarks auf TON-IOT . . . 5

3 Datensatz und Preprocessing **5**
3.1 TON-IOT Network Dataset . . . 5
3.2 Datenaufteilung . . . 5
3.3 Feature Engineering und Preprocessing . . . 6
3.3.1 Feature Elimination . . . 6
3.3.2 Context-Aware Imputation . . . 6
3.3.3 Skalierung und Encoding . . . 6
3.4 Label-Verteilung . . . 7

4 Methodik und Experimentelles Setup **7**

4.1 SVM-Modellvarianten . . . 7
4.1.1 SVC mit RBF-Kernel . . . 7
4.1.2 LinearSVC . . . 8
4.1.3 SGDClassifier . . . 8
4.2 Hyperparameter-Optimierung . . . 8
4.3 Resource Monitoring . . . 9
4.3.1 Zeitmetriken . . . 9
4.3.2 Speicherverbrauch . . . 9
4.3.3 CPU-Auslastung . . . 9
4.3.4 Energieverbrauch . . . 9
4.4 Evaluationsmetriken . . . 10
4.5 Experimentelles Setup . . . 10

5 Experimentelle Ergebnisse 10
5.1 Hyperparameter-Optimierung . . . 10
5.2 Performanz-Metriken auf Validation Set . . . 11
5.3 Confusion Matrices . . . 11
5.4 Ressourcenverbrauch . . . 12
5.5 Test Set Evaluation (SVC) . . . 13

6 Diskussion 13
6.1 Vergleich der SVM-Varianten . . . 13
6.2 Vergleich mit Literatur-Benchmarks . . . 14
6.3 Deployment-Szenarien . . . 14
6.4 Limitationen und zukünftige Arbeiten . . . 15
6.4.1 Limitationen . . . 15
6.4.2 Zukünftige Arbeiten . . . 15
6.5 Integration mit Projektpartner-Ergebnissen . . . 15

7 Zusammenfassung und Ausblick 16

Literatur 17

1 Einleitung

1.1 Motivation und Problemstellung

Mit der zunehmenden Digitalisierung und Vernetzung kritischer Infrastrukturen steigt auch die Bedrohung durch Cyberangriffe. Intrusion Detection Systeme (IDS) sind ein zentraler Bestandteil moderner Sicherheitsarchitekturen und dienen der Erkennung verdächtiger Aktivitäten in Computernetzwerken. Während traditionelle regelbasierte IDS-Ansätze auf bekannten Angriffsmustern basieren, ermöglichen Machine-Learning-Verfahren die Erkennung neuartiger Angriffe durch Anomalieerkennung. Eine besondere Herausforderung in diesem Kontext ist die Unbalanciertheit der Daten. Da der Normalzustand von Netzwerken ist, nicht angegriffen zu werden, sind Angriffsdaten selten. Gleichzeitig ist die Konsequenz einer verpassten Attacke (False Negative) deutlich schwerwiegender als ein Fehlalarm (False Positive). Dies erfordert spezialisierte Evaluationsmetriken und Optimierungsstrategien, die Recall über Precision priorisieren. Neben der Detektionsqualität gewinnt die Ressourceneffizienz zunehmend an Bedeutung. Mit dem Trend zu Edge Computing und IoT-Geräten müssen IDS-Algorithmen nicht nur akkurat, sondern auch ressourcenschonend sein. Daher ist die Frage nach einem geeigneten Trade-off zwischen Performanz und Effizienz von Relevanz.

1.2 Zielsetzung der Arbeit

Das Ziel dieser Projektarbeit ist ein umfassender Vergleich verschiedener Machine-Learning-Algorithmen für Intrusion Detection hinsichtlich:

1. **Detektionsqualität:** Evaluation mittels F1-Score, F2-Score, Precision, Recall und Accuracy
2. **Ressourceneffizienz:** Messung von Trainingszeit, Inferenzzeit, Speicherverbrauch und Energieverbrauch
3. **Praktikabilität:** Bewertung der Eignung für verschiedene Deployment-Szenarien (leistungsstarke Server vs. ressourcenbeschränkte Edge-Geräte)

Der vorliegende Bericht fokussiert auf SVM-basierte Ansätze (SVC, LinearSVC, SGD-Classifier) auf dem TON-IOT-Datensatz. Die Ergebnisse werden später mit den von Projektpartnern untersuchten Isolation Forest und Boosting-Verfahren zu einem Gesamtvergleich zusammengeführt.

1.3 Aufbau der Arbeit

Die Arbeit ist wie folgt strukturiert: Kapitel 2 gibt einen Überblick über verwandte Arbeiten und den Stand der Forschung. Kapitel 3 beschreibt den verwendeten TON-IOT-Datensatz und die Preprocessing-Schritte. Die methodischen Grundlagen und experimentellen Ansätze werden in Kapitel 4 erläutert. Kapitel 5 präsentiert die experimentellen Ergebnisse, gefolgt von einer kritischen Diskussion in Kapitel 6. Kapitel 7 fasst die Erkenntnisse zusammen und gibt einen Ausblick auf zukünftige Arbeiten.

2 Related Work und Theoretische Grundlagen

2.1 Intrusion Detection und Machine Learning

Intrusion Detection Systeme lassen sich grundsätzlich in signaturbasierte und anomaliebasierte Ansätze unterteilen. Während signaturbasierte Systeme bekannte Angriffsmuster erkennen, identifizieren anomaliebasierte Systeme Abweichungen vom normalen Netzwerkverhalten. Machine-Learning-Verfahren haben sich als vielversprechend für die Anomalieerkennung erwiesen, da sie in der Lage sind, komplexe Muster in hochdimensionalen Daten zu erkennen [1].

2.2 Der TON-IOT-Datensatz

Der TON-IOT-Datensatz [2] ist ein umfangreicher Benchmark für IDS-Algorithmen, der sowohl normale Netzwerkaktivitäten als auch verschiedene Angriffstypen enthält. Im Gegensatz zu älteren Datensätzen wie KDD Cup 99 oder NSL-KDD enthält TON-IOT moderne IoT-spezifische Angriffsszenarien und ist durch seine Größe und Diversität besonders für das Training komplexer ML-Modelle geeignet. Der Datensatz umfasst 42 Features, die aus Netzwerk-Traffic extrahiert wurden, darunter:

- **Verbindungsinformationen:** Source/Destination IP, Ports, Protokoll
- **Zeitliche Merkmale:** Verbindungsdauer, Paketintervalle
- **Statistiken:** Anzahl Bytes/Pakete, Flag-Zustände
- **Protokoll-spezifische Features:** DNS, HTTP, SSL-Attribute

Eine besondere Herausforderung ist die starke Unbalanciertheit: Ca. 76% der Samples sind als Angriffe gekennzeichnet, wobei jedoch verschiedene Angriffstypen sehr unterschiedlich häufig vorkommen.

2.3 Support Vector Machines für IDS

Support Vector Machines (SVM) haben sich als effektive Methode für binäre Klassifikationsprobleme etabliert. Das grundlegende Prinzip besteht darin, eine optimale Hyperebene zu finden, die die Klassen im Featureraum trennt [3]. Für nicht-linear separierbare Probleme können Kernel-Funktionen verwendet werden, um die Daten in einen höherdimensionalen Raum zu transformieren. Für IDS-Anwendungen sind besonders folgende SVM-Varianten relevant:

- **SVC (Support Vector Classifier):** Klassische SVM-Implementierung mit verschiedenen Kernel-Optionen (linear, polynomial, RBF). Bietet hohe Flexibilität, skaliert aber schlecht mit der Datensatzgröße $O(n^2)$ bis $O(n^3)$.
- **LinearSVC:** Optimiert für lineare Kernel, verwendet eine effizientere Optimierungsmethode (liblinear). Skaliert besser auf große Datensätze ($O(n)$), aber eingeschränkt auf lineare Entscheidungsgrenzen.

- **SGDClassifier:** Implementiert Stochastic Gradient Descent für lineare SVM. Besonders effizient für sehr große Datensätze, benötigt aber sorgfältige Hyperparameter-Tuning.

2.4 Evaluationsmetriken für Sicherheitsanwendungen

Für IDS-Systeme sind spezialisierte Evaluationsmetriken erforderlich, die die unterschiedliche Kritikalität von Fehlern berücksichtigen:

$$\text{F_}\beta\text{-Score} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \tag{1}$$

Der F2-Score ($\beta = 2$) gewichtet Recall doppelt so stark wie Precision und ist daher für Sicherheitsanwendungen besonders geeignet, da verpasste Angriffe (False Negatives) schwerwiegender sind als Fehlalarme (False Positives) [4].

2.5 Benchmarks auf TON-IOT

Uddin et al. [4] präsentierten einen umfassenden Benchmark verschiedener ML-Algorithmen auf TON-IOT, darunter Random Forest, Isolation Forest und usfAD. Besonders auffällig waren die schwachen F1-Scores von Isolation Forest-Methoden, was auf die Unbalanciertheit und Komplexität des Datensatzes zurückgeführt wurde. Belarbi et al. [3] untersuchten die Übertragbarkeit von IDS-Modellen auf neue Datensätze und betonten die Bedeutung von Feature Engineering und domänenspezifischem Preprocessing. Ihre Arbeit zeigt, dass generische ML-Ansätze ohne Anpassung oft suboptimale Ergebnisse liefern.

3 Datensatz und Preprocessing

3.1 TON-IOT Network Dataset

Für diese Arbeit wurde der *Train-Test Network* Teil des TON-IOT-Datensatzes verwendet, der 211.043 Samples mit 42 Features und 2 Labels (binär: normal/attack, kategorial: Angriffstyp) enthält. Die Daten wurden von Moustafa [2] in einer kontrollierten Testumgebung mit realistischen IoT-Geräten und simulierten Angriffsszenarien erhoben.

3.2 Datenaufteilung

Die Aufteilung des Datensatzes erfolgte stratifiziert nach Angriffstyp, um eine repräsentative Verteilung in allen Teilmengen zu gewährleisten:

- **Training Set:** 70% (147.730 Samples) – Für Modell-Training
- **Validation Set:** 15% (31.656 Samples) – Für Hyperparameter-Tuning und Modellauswahl

- **Test Set:** 15% (31.657 Samples) – Wird bis zur finalen Evaluation nicht verwendet

Die stratifizierte Aufteilung stellt sicher, dass alle drei Sets die gleiche Verteilung von Angriffstypen aufweisen (ca. 76.31% Angriffe). Dies ist wichtig, um realistische Evaluationsergebnisse zu erhalten.

3.3 Feature Engineering und Preprocessing

Das Preprocessing wurde in einer scikit-learn Pipeline implementiert und umfasst folgende Schritte:

3.3.1 Feature Elimination

IP-Adressen und Ports wurden entfernt, um die Generalisierungsfähigkeit zu verbessern. Diese Features sind hochspezifisch für die Testumgebung und würden in realen Szenarien zu Overfitting führen. Ein Modell sollte Angriffsmuster anhand des Verhaltens erkennen, nicht anhand spezifischer IP-Adressen.

Codeblock 1: Feature Elimination

```
class FeatureEliminator(BaseEstimator, TransformerMixin):
def init(self, features\_to\_drop):
self.features\_to\_drop = features\_to\_drop
def transform(self, X):
return X.drop(columns=self.features\_to\_drop)
```

3.3.2 Context-Aware Imputation

Fehlende Werte wurden kontextabhängig behandelt:

- **Kategoriale Features:** Fehlende Werte werden mit `"` gefüllt (repräsentiert "nicht verfügbar")
- **DNS-spezifisch:** `dns_rcode` wird auf `"` gesetzt, wenn `service` $\neq$ `"dns"`
- **Kontinuierliche Features:** `"` wird durch 0 ersetzt und numerisch konvertiert

Diese domänenspezifische Imputation ist wichtiger als generische Methoden (z.B. Mean-Imputation), da Fehlwerte oft semantische Bedeutung haben (z.B. "DNS-Feature nicht anwendbar, weil kein DNS-Protokoll").

3.3.3 Skalierung und Encoding

Die Feature-Transformation erfolgt mittels `ColumnTransformer` mit drei Komponenten:

1. **RobustScaler** für kontinuierliche Features (11 Features): Robuster gegen Outliers als StandardScaler, da Median und IQR statt Mean/Std verwendet werden.

2. **OneHotEncoder** für kategoriale Features mit wenigen Ausprägungen (14 Features): Erstellt binäre Dummy-Variablen, wobei die erste Kategorie gedroppt wird (`drop='first'`), um Multikollinearität zu vermeiden.
3. **Binary Presence Encoder** für hochkardinalere Features (13 Features): Statt One-Hot-Encoding (würde zu Dimensionsexplosion führen) wird binär kodiert: 0 wenn fehlend/leer, 1 wenn vorhanden. Dies reduziert die Feature-Anzahl drastisch bei minimalem Informationsverlust.

Nach dem Preprocessing resultieren 79 Features aus ursprünglich 42, wobei 4 Features (IPs, Ports) entfernt wurden.

3.4 Label-Verteilung

Die Label-Verteilung zeigt eine klare Dominanz von Angriffen (76.31%), was für IDS-Datensätze unüblich ist (üblicherweise dominiert normaler Traffic). Innerhalb der Angriffe gibt es jedoch eine starke Unbalanciertheit zwischen verschiedenen Angriffstypen. Tabelle 1 zeigt die Verteilung der Top-5 Angriffstypen im Training Set. Diese

Tabelle 1: Label-Verteilung im Training Set (Top-5 Angriffstypen)

Angriffstyp	**Anzahl**	**Anteil (%)**
Normal	35.012	23.69
DDoS	48.234	32.65
Scanning	31.892	21.59
Password	15.447	10.45
XSS	8.921	6.04
Andere	8.224	5.58

Verteilung motiviert die Verwendung von `class_weight='balanced'` in den SVM-Modellen und die Wahl des F2-Scores als Optimierungsmetrik.

4 Methodik und Experimentelles Setup

4.1 SVM-Modellvarianten

Es wurden drei SVM-Varianten implementiert und verglichen:

4.1.1 SVC mit RBF-Kernel

Der Support Vector Classifier mit Radial Basis Function (RBF) Kernel ist die flexibelste Variante und kann nicht-lineare Entscheidungsgrenzen modellieren. Die RBF-Kernel-Funktion ist definiert als:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma|\mathbf{x}_i - \mathbf{x}_j|^2\right) \tag{2}$$

Hyperparameter:

- $C \in [0.1, 100]$: Regularisierungsparameter (log-uniform)
- $\gamma \in \{\text{scale}, \text{auto}\}$: Kernel-Koeffizient
- `class_weight='balanced'`: Automatische Gewichtung für unbalancierte Daten

4.1.2 LinearSVC

LinearSVC ist auf lineare Kernel spezialisiert und verwendet die liblinear-Bibliothek, die für große Datensätze effizienter ist. Die Optimierung erfolgt über:

$$\min_\mathbf{w}, b\frac{1}{2}|\mathbf{w}|^2 + C\sum_i = 1^n \max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)) \tag{3}$$

Hyperparameter:

- $C \in [0.001, 100]$: Regularisierungsparameter (log-uniform)
- `loss` $\in \{$`hinge`, `squared_hinge`$\}$: Verlustfunktion
- `dual='auto'`: Automatische Wahl zwischen Primal/Dual basierend auf Datensatzgröße

4.1.3 SGDClassifier

Der Stochastic Gradient Descent Classifier approximiert eine lineare SVM durch iterative Optimierung mit Mini-Batches. Dies ist besonders effizient für sehr große Datensätze:

$$\mathbf{w}_t + 1 = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t, \mathbf{x}_i, y_i) \tag{4}$$

Hyperparameter:

- $\alpha \in [10^{-5}, 0.1]$: Learning rate / Regularisierung (log-uniform)
- `loss` $\in \{$`hinge`, `log_loss`$\}$: Verlustfunktion
- `penalty` $\in \{$`l2`, `l1`, `elasticnet`$\}$: Regularisierung
- `early_stopping=True`: Stoppt Training bei Stagnation

4.2 Hyperparameter-Optimierung

Die Hyperparameter-Optimierung erfolgte mittels Optuna [5], einem Bayesianischen Optimierungsframework. Optuna verwendet den Tree-structured Parzen Estimator (TPE) als Sampling-Strategie, der effizienter ist als Grid Search oder Random Search.
Optimierungsprozess:

1. 5-fold stratifizierte Cross-Validation auf dem Training Set
2. Optimierungsmetrik: F2-Score (priorisiert Recall)
3. Anzahl Trials: 200 (oder Timeout nach 3600s)
4. Parallele Evaluation mit `n_jobs=-1`

Die Wahl des F2-Scores als Optimierungskriterium reflektiert die Anforderungen von Sicherheitsanwendungen: Ein verpasster Angriff (False Negative) ist kritischer als ein Fehlalarm (False Positive). Der F2-Score gewichtet Recall doppelt so stark wie Precision.

4.3 Resource Monitoring

Um einen umfassenden Vergleich zu ermöglichen, wurden systematisch Ressourcenverbrauchsdaten erfasst:

4.3.1 Zeitmetriken

- **Wall Time:** Gesamte verstrichene Zeit (inkl. I/O, System-Overhead)
- **Process Time:** Reine CPU-Zeit des Python-Prozesses (exkl. I/O)
- **Throughput:** Samples pro Sekunde bei Inferenz

4.3.2 Speicherverbrauch

Mittels `tracemalloc` wurde der Peak Memory Usage gemessen. Dies ist besonders relevant für Edge-Deployment, wo Speicher limitiert ist.

4.3.3 CPU-Auslastung

Die CPU-Auslastung wurde während Training/Inferenz periodisch (alle 2s) gesampelt mittels `psutil`. Dies gibt Aufschluss über Parallelisierungseffizienz und System-Load.

4.3.4 Energieverbrauch

Der Energieverbrauch wurde mittels CodeCarbon [6] geschätzt. CodeCarbon trackt CPU- und GPU-Auslastung und berechnet basierend auf TDP und Laufzeit den Energieverbrauch in kWh sowie CO_2-Emissionen.

4.4 Evaluationsmetriken

Die Evaluation erfolgte auf dem Validation Set mit folgenden Metriken:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

$$\text{F2-Score} = 5 \cdot \frac{\text{Precision} \cdot \text{Recall}}{4 \cdot \text{Precision} + \text{Recall}} \tag{8}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{9}$$

Zusätzlich wurden sicherheitsspezifische Metriken berechnet:

- **Detection Rate:** Anteil erkannter Angriffe (= Recall)
- **Missed Attacks:** Absolute Anzahl verpasster Angriffe (= False Negatives)
- **False Alarm Rate:** Anteil fälschlicherweise als Angriff klassifizierter normaler Samples

4.5 Experimentelles Setup

Hardware:

- CPU: AMD Ryzen 7 7840HS (8 Cores, 16 Threads, 4.7 GHz)
- RAM: 64 GB DDR5
- OS: Windows 11
- Python: 3.13.7

Software:

- scikit-learn 1.6.0
- optuna 4.1.0
- codecarbon 2.7.3

Der Code ist verfügbar unter: `[GitHub-Link einfügen]`

5 Experimentelle Ergebnisse

5.1 Hyperparameter-Optimierung

Die Hyperparameter-Optimierung mittels Optuna ergab folgende Best Parameters:

Tabelle 2: Optimale Hyperparameter nach Optuna-Tuning

Modell	Parameter	Wert	CV F2-Score
SVC	C gamma	6.251 scale	0.9847
LinearSVC	C loss	0.873 squared_hinge	0.9821
SGDClassifier	alpha loss penalty	0.00152 hinge l2	0.9654

Interessanterweise erreichte SVC mit RBF-Kernel den höchsten CV F2-Score, was die Vorteile nicht-linearer Entscheidungsgrenzen demonstriert. LinearSVC liegt nur knapp dahinter, während SGDClassifier ca. 2% schlechter abschneidet.

5.2 Performanz-Metriken auf Validation Set

Tabelle 3 zeigt die detaillierten Evaluationsergebnisse auf dem Validation Set (31.656 Samples):

Tabelle 3: Performanz-Metriken auf Validation Set

Modell	F1	F2	Precision	Recall	Accuracy
SVC	0.8657	0.9416	0.7633	1.0000	0.7633
LinearSVC	0.9686	0.9798	0.9505	0.9874	0.9511
SGDClassifier	0.6939	0.6717	0.7345	0.6576	0.5573

Interpretation:

- **SVC:** Erreicht perfekte Recall (100%), d.h. kein einziger Angriff wird verpasst. Der Preis dafür sind 7.492 False Positives (Fehlalarme), was zu niedriger Precision (76.33%) führt. Dies ist ein typischer Precision-Recall-Trade-off.
- **LinearSVC:** Bietet den besten Balance mit F2-Score 0.9798. Verpasst nur 304 von 24.157 Angriffen (1.26% Miss Rate) bei deutlich weniger Fehlalarmen (1.243) als SVC. Dies ist der praktikabelste Ansatz.
- **SGDClassifier:** Zeigt signifikant schlechtere Performanz mit nur 65.76% Detection Rate. Dies liegt vermutlich an suboptimaler Konvergenz trotz Hyperparameter-Tuning. SGD ist sensitiver gegenüber Hyperparametern als andere SVM-Varianten.

5.3 Confusion Matrices

Tabelle 4 zeigt die Confusion Matrices für alle drei Modelle:

Tabelle 4: Confusion Matrices (Validation Set)

Modell	**Predicted Normal**		**Predicted Attack**	
	TN	FN	FP	TP
SVC	8	0	7.492	24.157
LinearSVC	6.257	304	1.243	23.853
SGDClassifier	1.758	8.271	5.742	15.886

Die Confusion Matrices verdeutlichen die unterschiedlichen Trade-offs:

- SVC minimiert FN (0) auf Kosten von FP (7.492)
- LinearSVC balanciert FN (304) und FP (1.243)
- SGDClassifier hat sowohl hohe FN (8.271) als auch hohe FP (5.742)

5.4 Ressourcenverbrauch

Tabelle 5 zeigt die gemessenen Ressourcenverbrauchsdaten:

Tabelle 5: Ressourcenverbrauch (Training auf 147.730 Samples, Inferenz auf 31.656 Samples)

Modell	**Train Time (s)**	**Inference Time (s)**	**Throughput (samples/s)**	**Peak RAM (MB)**	**Energy (kWh)**
SVC	1.247	257.81	123	0.92	0.0275
LinearSVC	89.32	1.65	19.132	0.85	0.0034
SGDClassifier	12.45	1.68	18.789	0.85	0.0015

Interpretation:

- **Training:** LinearSVC hat die längste Trainingszeit (89.32s), da Optuna 200 Trials durchführt. SVC ist trotz RBF-Kernel schneller (1.247s), da hier nur wenige Trials nötig waren bis Timeout. SGDClassifier ist am schnellsten (12.45s).
- **Inferenz:** SVC ist extrem langsam (257.81s für 31.656 Samples = 123 samples/s). Dies liegt an der Komplexität der RBF-Kernel-Berechnung für alle Support Vectors. LinearSVC und SGD sind ca. 150x schneller (1.65-1.68s = 19k samples/s).
- **Speicher:** Alle Modelle haben ähnlichen Peak Memory (0.85-0.92 MB), da scikit-learn effiziente Implementierungen nutzt.
- **Energie:** SVC verbraucht am meisten Energie (0.0275 kWh) aufgrund langer Inferenzzeit. LinearSVC (0.0034 kWh) und SGD (0.0015 kWh) sind deutlich energieeffizienter.

5.5 Test Set Evaluation (SVC)

Zur finalen Evaluation wurde das beste Modell (SVC basierend auf F2-Score) auf dem Test Set getestet. Tabelle 6 zeigt die Ergebnisse:

Tabelle 6: Test Set Evaluation (SVC, 31.657 Samples)

Metrik	**Wert**	**Confusion Matrix**		
F1-Score	0.8657	Predicted Normal	Predicted Attack	
F2-Score	0.9416	True Normal	8	7.492
Precision	0.7633	True Attack	0	24.157
Recall	1.0000			
Accuracy	0.7633			
Detection Rate	100.00%			
Missed Attacks	0 / 24.157			
False Alarms	7.492			

Die Test-Ergebnisse bestätigen die Validation-Resultate: SVC erreicht perfekte Angriffserkennung (100% Recall) bei moderater Precision. Dies zeigt, dass keine Overfitting-Probleme vorliegen.

6 Diskussion

6.1 Vergleich der SVM-Varianten

Die Experimente zeigen deutliche Trade-offs zwischen den drei SVM-Varianten: **SVC mit RBF-Kernel** bietet die höchste Detektionsqualität (100% Recall, F2-Score 0.9416) und ist damit die sicherste Option aus Sicherheitsperspektive. Kein einziger Angriff wird verpasst. Der Preis dafür sind:

- Hohe False Positive Rate (fast alle normalen Samples werden als Angriff klassifiziert)
- Extrem langsame Inferenz (257s für 31k Samples = 155x langsamer als LinearSVC)
- Höchster Energieverbrauch (0.0275 kWh)

Eignung: Nur für Szenarien geeignet, wo False Positives akzeptabel sind und Rechenressourcen unbegrenzt (z.B. Offline-Analyse, Cloud-Deployment mit manueller Nachverifikation). **LinearSVC** bietet den besten Kompromiss:

- Sehr gute Detektionsqualität (98.74% Recall, F2-Score 0.9798)
- Nur 304 verpasste Angriffe (1.26% Miss Rate)
- Deutlich weniger Fehlalarme als SVC (1.243 vs. 7.492)
- 155x schnellere Inferenz als SVC

- Energieeffizient (0.0034 kWh)

Eignung: Optimal für praktische Deployments, sowohl auf Servern als auch Edge-Geräten. Der minimale Recall-Verlust (1.26%) ist vertretbar angesichts der massiven Effizienzgewinne. **SGDClassifier** zeigt enttäuschende Ergebnisse:

- Nur 65.76% Recall (34.24% Miss Rate!)
- Trotzdem viele False Positives (5.742)
- Zwar schnell, aber unbrauchbar für Sicherheitsanwendungen

Ursachen: SGD ist bekanntermaßen sensitiv gegenüber Hyperparametern und Datensatz-Charakteristiken. Die Unbalanciertheit und hohe Dimensionalität von TON-IOT scheinen SGD zu benachteiligen. Möglicherweise wären mehr Tuning-Trials oder adaptive Learning Rates (z.B. AdaGrad) hilfreich.

6.2 Vergleich mit Literatur-Benchmarks

Uddin et al. [4] berichten für ihre usfAD-Methode F1-Scores im Bereich 0.85-0.95 auf TON-IOT. Unser LinearSVC (F1: 0.9686) ist vergleichbar oder besser, bei deutlich geringerer Komplexität. Belarbi et al. [3] betonen die Wichtigkeit von Feature Engineering. Unsere domänenspezifische Imputation und Binary Presence Encoding scheinen effektiv zu sein, da wir gute Ergebnisse mit klassischen ML-Methoden erreichen. Die in [4] beobachteten schwachen Isolation Forest Ergebnisse werden von unseren Projektpartnern weiter untersucht. Ein späterer Vergleich wird zeigen, ob ähnliche Probleme auftreten.

6.3 Deployment-Szenarien

Basierend auf den Ergebnissen lassen sich folgende Empfehlungen für verschiedene Deployment-Szenarien ableiten: **Edge Computing / IoT-Geräte:**

- **Empfohlen:** LinearSVC
- Begründung: Schnelle Inferenz (19k samples/s), geringer Speicherbedarf, energieeffizient
- Trade-off: 1.26% Missed Attacks akzeptabel

Cloud / Datacenter:

- **Empfohlen:** LinearSVC oder SVC (je nach Anforderungen)
- LinearSVC für hohen Durchsatz bei guter Qualität
- SVC für maximale Sicherheit bei unbegrenzten Ressourcen

Hybrid-Ansatz:

- LinearSVC auf Edge für Echtzeit-Filterung
- Verdächtige Samples zur Cloud für SVC-Nachverifikation

- Beste Balance zwischen Latenz und Accuracy

6.4 Limitationen und zukünftige Arbeiten

6.4.1 Limitationen

1. **Datensatz-spezifisch:** Die Ergebnisse sind spezifisch für TON-IOT. Generalisierung auf andere Netzwerke oder neue Angriffstypen ist unklar.
2. **Simulated Environment:** TON-IOT wurde in kontrollierter Umgebung erstellt. Reale Netzwerke haben andere Charakteristiken (mehr Rauschen, variablere Angriffsformen).
3. **Binary Classification:** Wir haben nur binär klassifiziert (normal vs. attack). Multi-Class-Klassifikation der Angriffstypen wäre informativer für Security-Teams.
4. **Static Model:** Das Modell lernt nicht kontinuierlich. Neue Angriffstypen erfordern Retraining.
5. **Feature Drift:** Netzwerk-Traffic ändert sich über Zeit. Regelmäßige Re-Evaluation wäre nötig.

6.4.2 Zukünftige Arbeiten

1. **Multi-Class-Klassifikation:** Erweiterung auf Angriffstyp-Erkennung
2. **Ensemble-Methoden:** Kombination von LinearSVC mit anderen Algorithmen (z.B. Random Forest, Isolation Forest von Projektpartnern)
3. **Transfer Learning:** Training auf TON-IOT, Testing auf anderen Datensätzen (z.B. Gotham Dataset [3])
4. **Online Learning:** Implementierung von inkrementellem Learning für adaptive IDS
5. **Explainability:** SHAP/LIME-Analysen um zu verstehen, welche Features für Angriffserkennung wichtig sind
6. **Real-World-Testing:** Deployment in realer Netzwerkumgebung mit Monitoring von False Positive/Negative Rates

6.5 Integration mit Projektpartner-Ergebnissen

Die finale Projektarbeit wird die hier präsentierten SVM-Ergebnisse mit den von Projektpartnern untersuchten Isolation Forest und Boosting-Verfahren kombinieren. Geplant ist:

- Gemeinsame Evaluation aller Algorithmen auf identischem Test Set
- Vergleich auf zwei Hardware-Plattformen:
  - High-Performance: AMD Ryzen 7 7840HS (wie hier verwendet)

  - Low-Power: Raspberry Pi 4 oder ähnliches Edge-Device
- Unified Benchmark-Tabelle mit allen Metriken
- Diskussion von Ensemble-Strategien (z.B. Voting zwischen SVM, Isolation Forest, XGBoost)

Dieser umfassende Vergleich wird wertvolle Insights für die praktische Auswahl von IDS-Algorithmen liefern.

7 Zusammenfassung und Ausblick

Diese Arbeit untersuchte drei Support Vector Machine Varianten (SVC mit RBF-Kernel, LinearSVC, SGDClassifier) für Anomalieerkennung im TON-IOT Network Datensatz. Der Fokus lag auf einem ganzheitlichen Vergleich, der neben klassischen Performanz-Metriken auch Ressourceneffizienz (Zeit, Speicher, Energie) berücksichtigte. **Hauptergebnisse:**

1. **Beste Detektionsqualität:** SVC mit RBF-Kernel erreicht perfekte Angriffserkennung (100% Recall, F2-Score 0.9416), ist aber mit 257s Inferenzzeit für 31k Samples praktisch nicht einsetzbar.
2. **Bester Trade-off:** LinearSVC bietet hervorragende Detektionsqualität (98.74% Recall, F2-Score 0.9798) bei 155x schnellerer Inferenz und deutlich geringerem Energieverbrauch. Dies ist die empfohlene Wahl für praktische Deployments.
3. **SGDClassifier ungeeignet:** Trotz Hyperparameter-Tuning erreicht SGD nur 65.76% Recall und ist damit für Sicherheitsanwendungen unbrauchbar.
4. **Deployment-Empfehlungen:** LinearSVC eignet sich sowohl für Edge-Geräte (schnell, energieeffizient) als auch Cloud-Deployments (hoher Durchsatz). Für maximale Sicherheit bei unbegrenzten Ressourcen kann SVC mit RBF-Kernel verwendet werden.

Beitrag zur Projektarbeit: Diese Arbeit stellt den SVM-Teil einer umfassenden Benchmark-Studie dar. Die entwickelte Pipeline (Feature Engineering, Hyperparameter-Tuning, Resource Monitoring) ist modular aufgebaut und wird von Projektpartnern für Isolation Forest und Boosting-Verfahren wiederverwendet. Die finale Projektarbeit wird alle Algorithmen in einem unified Framework vergleichen und auf verschiedenen Hardware-Plattformen evaluieren. **Ausblick:** Zukünftige Arbeiten sollten sich auf Multi-Class-Klassifikation (Angriffstyp-Erkennung), Ensemble-Methoden und Transfer Learning auf neue Datensätze fokussieren. Besonders vielversprechend ist die Kombination von LinearSVC (für schnelle Filterung) mit komplexeren Modellen (für Nachverifikation) in einem Hybrid-Deployment. Die hier erzielten Ergebnisse demonstrieren, dass klassische Machine-Learning-Methoden wie SVM bei sorgfältigem Feature Engineering und Hyperparameter-Tuning State-of-the-Art Performanz für IDS-Anwendungen erreichen können, ohne die Komplexität von Deep Learning.

Literatur

[1] Mohammadreza Rabbani, Jiacheng Gui, Farhad Nejati, Zhiyuan Zhou, Abraham Kaniyamattam, Muhammad Mirani, Girish Piya, Ivan Opushnyev, Rongxing Lu, and Ali A. Ghorbani. Device identification and anomaly detection in iot environments. *IEEE Internet of Things Journal*, 2024.

[2] Nour Moustafa. A new distributed architecture for evaluating ai-based security systems at the edge: Network ton_iot datasets. *Sustainable Cities and Society*, 72:102994, 2021.

[3] Othmane Belarbi, Theodoros Spyridopoulos, Eirini Anthi, Omer Rana, Pietro Carnelli, and Amir Khan. Gotham dataset 2025: A reproducible large-scale iot network dataset for intrusion detection and security research. *arXiv preprint arXiv:2502.03134*, 2025.

[4] Md Ashraf Uddin, Sunil Aryal, Mohamed Reda Bouadjenek, Muna Al-Hawawreh, and Md Abul Talukder. usfad-based effective unknown attack detection focused ids framework. *Scientific Reports*, 14:29103, 2024.

[5] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. `https://optuna.org/`, 2019.

[6] Code Carbon Team. Codecarbon: Track and reduce co2 emissions from your computing. `https://codecarbon.io/`, 2021.